

题目链接

UVa12419 - Heap Manager

题意

内存以内存单元为基本单位，每个内存单元用一个固定的整数作为标识，称为地址。地址从0开始连续排列，地址相邻的内存单元被认为是逻辑上连续的。我们把从地址*i*开始的*s*个连续的内存单元称为首地址为*i*长度为*s*的地址片。

运行过程中有若干进程需要占用内存，对于每个进程有一个申请时刻*t*，需要内存单元数*m*及运行时间*p*。在运行时间*p*内（即时刻开始，*t*+*p*时刻结束），这*m*个被占用的内存单元不能再被其他进程使用。

假设在*t*时刻有一个进程申请*m*个单元，且运行时间为*p*，则：

（1）若*t*时刻内存中存在长度为*m*的空闲地址片，则系统将这*m*个空闲单元分配给该进程。若存在多个长度为*m*的空闲地址片，则系统将首地址最小的那个空闲地址片分配给该进程。

（2）如果*t*时刻不存在长度为*m*的空闲地址片，则该进程被放入一个等待队列。对于处于等待队列队头的进程，只要在任一时刻，存在长度为*m*的空闲地址片，系统马上将该进程取出队列，并为其分配内存单元。注意，在进行内存分配处理过程中，处于等待队列队头的进程的处理优先级最高，队列中的其他进程不能先于队头进程被处理。

现在给出一些描述进程的数据（内存总量≤1e9，进程数≤200000），请编写一程序模拟系统分配内存的过程。

分析

利用优先级队列和普通队列即可模拟系统分配内存的过程：已经分配了内存的进程放入优先级队列，按结束时刻排序；申请时刻*t*未分配到内存的进程放入等待队列。

内存的分配与释放通过线段树来做，本题交代的内存总量是1e9级别的，因此需要动态开点的线段树来处理。

线段树动态开点需要分析单次操作涉及到的区间结点数上限，再乘上不同操作次数上限，推算出最大结点数。当然，也可以写动态内存分配与释放的指针版动态开点线段树模板，就不用分析这些了。

如果区间范围是*N*，则单次点修改操作涉及到的区间结点数上限是 $\lceil \log_2 N \rceil + 1$ ，单次区间修改操作涉及到的区间结点数上限是 $4 \lceil \log_2 N \rceil$ 。[知乎](#)上有博主将这个分析过程讲得很详细，还画了图示意。

单次操作涉及到的区间结点数上限，乘上不同操作次数上限，就得到了动态开点的结点数上限，为什么是乘上不同操作次数上限呢？因为同一个区间可能存在多次操作，实际上可以去重，比如本题进程数上限是200000，每个进程分配内存和释放内存都是在同一个区间上操作的，所以不同操作次数上限是200000而不是400000。

再说一下处理本题内存查询的方法，每个区间结点需要维护三个数据：区间内全0前缀长度*a*、区间内全0后缀长度*b*、区间内全0最大长度*c*。利用这三个数据可以实现本题查询需求。

AC代码

```
#include <cstdio>
#include <queue>
using namespace std;

#define N 200050
int a[120*N], b[120*N], c[120*N], z[120*N], lc[120*N], rc[120*N], n, g, x; struct {int i, m; long long t;} q[N];

struct node {
    long long t; int s, e;
    bool operator< (const node& rhs) const {
        return t > rhs.t;
    }
} r;

int query(int o, int l, int r, int s) {
    if (s == 0 || a[o] >= s) return l;
    if (c[o] < s) return 0;
    int m = (l+r)>>1, cl = lc[o] ? c[lc[o]] : m-l+1, bl = lc[o] ? b[lc[o]] : m-l+1, cr = rc[o] ? c[rc[o]] : r-m;
    if (cl >= s) return lc[o] ? query(lc[o], l, m, s) : l;
    if (bl + (rc[o] ? a[rc[o]] : r-m) >= s) return m-bl+1;
    return cr >= s ? (rc[o] ? query(rc[o], m+1, r, s) : m+1) : 0;
}

void pushdown(int o, int l, int r) {
    if (z[o] >= 0) {
        a[o] = b[o] = c[o] = z[o] ? 0 : r-l+1;
    } else {
        int m = (l+r)>>1, bl = lc[o] ? b[lc[o]] : m-l+1, ar = rc[o] ? a[rc[o]] : r-m;
        a[o] = lc[o] && a[lc[o]] <= m-l ? a[lc[o]] : m-l+1 + (rc[o] ? a[rc[o]] : r-m);
        b[o] = rc[o] && b[rc[o]] < r-m ? b[rc[o]] : r-m + (lc[o] ? b[lc[o]] : m-l+1);
        c[o] = max(bl+ar, max(lc[o] ? c[lc[o]] : m-l+1, rc[o] ? c[rc[o]] : r-m));
    }
}

void check(int &o, int l, int r) {
    if (o) return;
    o = ++x; a[o] = b[o] = c[o] = r-l+1; z[o] = lc[o] = rc[o] = 0;
}

void occ(int o, int l, int r, int x, int y, bool f) {
    if (x > y) return;
    if (l>=x && r<=y) {
        z[o] = f;
    } else {
        int m = (l+r)>>1; check(lc[o], l, m); check(rc[o], m+1, r);
        if (z[o] >= 0) z[lc[o]] = z[rc[o]] = z[o], z[o] = -1;
        x <= m ? occ(lc[o], l, m, x, y, f) : pushdown(lc[o], l, m);
        y > m ? occ(rc[o], m+1, r, x, y, f) : pushdown(rc[o], m+1, r);
    }
    pushdown(o, l, r);
}

void solve() {
    a[1] = b[1] = c[1] = n; z[x = 1] = lc[1] = rc[1] = 0;
    int m, i = 0, a, head = 0, tail = 0; long long ans = 0, t, p; priority_queue<node> s;
    while (scanf("%lld%lld", &t, &m, &p) && (t || m || p) && ++i) {
        while (!s.empty() && s.top().t <= t) {
            for (ans = s.top().t; !s.empty() && s.top().t == ans; s.pop()) occ(1, 1, n, s.top().s, s.top().e, 0);
            while (head < tail) {
                if (!(a = query(1, 1, n, q[head].m))) break;
                if (g) printf("%lld %d %d\n", ans, q[head].i, a-1);
                r.t = ans + q[head].t; r.s = a; r.e = a+q[head++].m-1; occ(1, 1, n, a, r.e, 1); s.push(r);
            }
        }
        if (a = query(1, 1, n, m)) {
            if (g) printf("%lld %d %d\n", t, i, a-1);
            r.t = t + p; r.s = a; r.e = a+m-1; occ(1, 1, n, a, r.e, 1); s.push(r);
        } else q[tail].i = i, q[tail].t = p, q[tail++].m = m;
    }
    while (!s.empty()) {
        for (ans = s.top().t; !s.empty() && s.top().t == ans; s.pop()) occ(1, 1, n, s.top().s, s.top().e, 0);
        while (head < tail) {
            if (!(a = query(1, 1, n, q[head].m))) break;
            if (g) printf("%lld %d %d\n", ans, q[head].i, a-1);
            r.t = ans + q[head].t; r.s = a; r.e = a+q[head++].m-1; occ(1, 1, n, a, r.e, 1); s.push(r);
        }
    }
    printf("%lld\n%d\n\n", ans, tail);
}

int main() {
    while (scanf("%d%d", &n, &g) == 2) solve();
    return 0;
}
```